AMB2064

Python Foundations – Activity Handouts

Activity 0.1 – Everyday Task as an Algorithm

Title: From Daily Routine to Algorithm

Instructions to Students

1. Choose one simple daily activity (examples: making tea, preparing to go to class, booking a cab).
2. Write the steps in clear, numbered points.
3. Make sure each step is:
   - Unambiguous
   - In correct order
   - Does not skip important details

Template

1. Activity chosen: Making Coffee in Hostel.
2. Write the steps:
   1. 1) Firstly Collect the required materials like Coffee powder,
   2. Kettle, Milk powder, water, Sugar, cup & Spoon
   3. 2) Select Suitable place to make Coffee (plug 3 pin)
   4. 3) Switch on (when Kettle plug is plugged in) Add water (1 cup)
   5. 4) And when fumes get start add 2 spoon Sugar (100 gee)
   6. 5) Later take cup in that Add 1½ spoon milk powder and 1 spoon Coffee powder ~~then~~
3. Check your algorithm:
   - Can a friend follow it without asking questions? Yes / No
   - If "No", refine the steps here:
     - _____

6) Lastly take spoon turn (any manner) it, my cup of Coffee is ready.

Title: Writing Pseudo-code with IF and REPEAT

Instructions

1. Take your steps from Activity 0.1.
2. Identify where decisions happen (IF/ELSE).
3. Identify where repetition happens (REPEAT / WHILE).
4. Rewrite your algorithm using the following keywords:
   - IF, ELSE, ELSE IF
   - REPEAT, WHILE, UNTIL

Template

1. Original activity: Making a cup of coffee in Hostel.
2. Pseudo-code version:

text

START

Making a cup of coffee in Hostel.

IF All ingredients are available THEN

I am able to make Coffee.

ELSE

I Can't prepare a Coffee.

ENDIF

REPEAT

_____

UNTIL _____

END

3. Reflection:
   - Where did you use IF/ELSE? Where Conditions are there.
   - Where did you use REPEAT/WHILE? _____

Activity 1.1 – Run and Edit Given Code

Title: First Python Script – Edit and Run

Instructions

1. Your teacher will give you a small Python script (example below).
2. Do NOT write new code first; only change text or numbers.
3. Run the program and observe the output.

Example Script

python

```
name = "Student"
age = 18
print("Hello,", name)
print("You are", age, "years old.")
```

Tasks

1. Change name and age to your own values.
   - name: Dharanesh. M.P
   - age: 23 (23)
2. Run the program. What is the output?

text

Output:

Hello, Dharanesh. M.P
You are 23 years old.

3. Change the printed message to:
   - "Welcome to Python, <your name>"

Write your edited code here:

python

Edited code:

Print ("welcome to python", name)
Print ("You are", age, "Years old.")

should be there

4. Reflection:
   - What changed in the output and why?

Hello replaced with welcome to python
— Because I given print input as "welcome to python"

Activity 1.2 – Predict–Then–Run

Title: Predict Program Output

Instructions

1. Look at each code snippet.
2. BEFORE running, write what you think the output will be.
3. Then run it and compare.

Snippet A

python

```
x = 5
y = 2
z = x + y * 3
print(z)
```

2×3

5+6.

- Predicted output: 11
- Actual output: ~~20~~ 11

~~7×3~~

~~5+2~~

Snippet B

python

```
a = 10
b = a - 3
a = a + 1
print(a, b)
```

- Predicted output: (11, 7)
- Actual output: (11, 7)

Reflection

- Where was your prediction wrong? Why?

Snippet A → predicted output uses BODMAS in problem where in Actual not specified in python input may be.

Activity 2.1 – Calculator in Steps

Title: From Word Problem to Code

Problem 1

A student has marks in 3 subjects. Find total and average.

1. Write the steps in plain language:
   1. 3 Subjects are maths, Science and Social Sciences.
   2. Student Scored in Maths 98, ~~out~~ Science 86 & Social Science 98
   3. Then the total marks equals maths + Science + Social Science
   4. Average marks equals Total marks / Total Subject.
2. Now convert to Python (fill in the blanks):

python

mark1 = 98

mark2 = 86

mark3 = 98

total = 282 94

average = 94

print("Total =", total)

print("Average =", average)

```
mark 1 = int (Input ("Enter mark 1 : Maths -")
mark 2 = int (Input ("Enter mark 2 : Science -")
mark 3 = int (Input ("Enter mark 3 : Social -")
Total = mark 1 + mark 2 + mark 3
avg = Total / 3
Print ("Total =", total)
Print ("Average =", avg)
```

3. Test with your own numbers and record output.

Total = 282 Average = 94

Activity 2.2 – Fix the Errors

Title: Debugging Simple Python

Instructions

Each snippet has an error.

1. Write what you think is wrong.
2. Correct the code.

Snippet 1

python

print("Hello World)

- Error: Syntax error. ~~Hola~~ error in line 1
- Corrected code: Print ("Hello world")

python

undefined

Snippet 2

Syntax error ; unterminated string literal.

python

```
a = 10
b = "5"
c = a + b
print(c)
```

- Error: b = "5", ~~do~~ error in line 2,
- Corrected code:

```
a = 10
b = 5
c = a+b
Print (c)
```

python

undefined

Reflection:

- What did you learn about syntax vs logic errors?

Syntax is alphabetical error (grammetical)
logic error is algorithm

A. File "_____", line 2
b = "5
Syntax error : unterminated string literal (detected at line 2)

Activity 3.1 – Rule-based Classification

Title: Grades with IF/ELSE

Rule

- If mark ≥ 90 : Grade A
- Else if mark ≥ 75 : Grade B
- Else : Grade C

Tasks

1. Draw the decision in simple text:
   - Condition 1: marks ≥ 90 : Grade A
   - Condition 2: 75 ≤ marks ≤ 90 : Grade B
2. Now write Python code:

python

```
mark = int(input("Enter mark: "))
if mark ≥ 90 :
    print("Grade A")
elif 75 ≤ mark ≤ 90 :
    print("Grade B")
else:
    print("Grade C")
```

3. Test with at least 3 marks and note outputs.

mark = 90.	mark = 75	mark = 51
output : Grade A	Grade B	Grade C.

Activity 3.2 – Mini Problem Bank

Title: Structured Problem Solving with IF/ELSE

For each question, fill the template.

Template

1. Restate the problem in one sentence:

   Odd or even numbers.

2. Conditions needed (bullets):

   - If N%2 = 0 : even
   - else odd.

3. Python code:

python

 # Your solution

```
# even or odd numbers.
~~for N in range 1 to N.~~
If N%2 =0
Print ("Even")
Else
Print ("odd")
```

4. Test cases and results:

   - Input: 14 Output: Even
   - Input: 7 Output: Odd

Use this template for:

- Odd or even number.
- Ticket discount if age < 12 or > 60.
- Check if a year is a leap year (simple rule).

Activity 4.1 – Sum with Loops

$\frac{n(n-1)}{2}$

Title: Sum from 1 to N

Steps

1. In plain language:
   - What is "sum from 1 to N"? $\frac{n(n+1)}{2}$
2. Write the algorithm:

1. Use formula $\frac{n(n+1)}{2}$ for all Natural number
2.
3. ranges from 1 to N
4.

3. Now code it:

python

```
N = int(input("Enter N: "))
total = 0
for i in range(1, n+1):
  total = total + n(n+1)/2
print("Sum =", total)
```

Sum with loops.
for natural numbers.
f = n(n+1)/2
Print ("f")

4. Try N = 5, 10, 100 and check results.

$\frac{5(5+1)}{2} \Rightarrow 15$ $\quad \frac{10(10+1)}{2} \Rightarrow 55$ $\quad \frac{100(100+1)}{2} \Rightarrow 5050$ $\quad \frac{101 \times 50}{5050}$

```
n = int (input ("Enter a positive integer(n):"))
if n<0:
    print ("please enter a positive integer.")
else:
    sum n=0
    for i in range (1, n+1):
        sum n += i
    Print (f"The sum of the first {n} natural numbers:{
```

Activity 4.2 – Star Patterns

Title: Loops for Patterns

Pattern (triangle)

```
*
**
***
****
```

1. Explain the pattern in words:

(N - *) ~~inc~~ No of natural number of stars printed in ~~Column~~ rows

2. Write code:

python

```
rows = int(input("Enter number of rows: "))
for i in range(1, rows + 1):
  print("*" * ____i____)
```

3. Try different rows and observe.

```
Row = int (Input ("Enter number of rows :"))
for i in range (1, rows +2):
    Print ("*" *i)
```

O/P — Enter the rows = 2

```
*
* * *
```

Activity 5.1 – Working with Lists

Title: Analyze Marks List

Given

python

marks = [56, 78, 90, 45, 88]

1. In words, write how to find:
   - Maximum mark: arrange the marks in ascending order. last one is max.
   - Average mark: Total marks divider with numbers of marks.
   - Count of marks ≥ 60: len ("marks ≥ 60") (or)
2. Now write Python:

python

marks = [56, 78, 90, 45, 88]

```
# Max
max_mark = 90 ("max"(marks))

# Average
total = 0
for m in marks:
    total = 352 (total + m)
average = 70.4 (total / len(marks))

# Count >= 60
count = 0
for m in marks:
    if m ≥ 60:
        count = 4 (len(count >= 60))

print("Max:", max_mark)
print("Average:", average)
print("Count >= 60:", count)
```

```
marks = [56, 78, 90, 45, 88]
max marks = max (marks)
Print ("Max marks:", max_marks)
min marks = min (marks)
Print ("min marks:", min_marks)
Total marks = sum (marks)
Print ("Total marks:", total_marks)
avg_marks = Total_marks / len (marks)
Print ("Average marks:", avg_marks)
Count_high marks = 0
for m in marks:
    if m >= 60:
        Count_high_marks += 1
Print ("Number of marks >= 60:", Count_high marks)
```

56
78
90
45
88
352 / 5 = 70.4

Activity 5.2 – Choosing an Approach

Title: Which Method is Better?

Problem

Given a list of 1000 numbers, find how many are positive.

1. Approach A (loop) – describe in words:
Initialize a counter to 0, then iterate through each number in the list.

2. Approach B (using built-in functions or list comprehension) – describe:
Use len to count numbers.

3. Which is easier to understand for you now? Why?
Approach B is easier, because len is counting itself. where Ap

4. Implement one approach in Python:

python

```
numbers = [10, -3, 5, 0, 8, -2] # example

# Your code
Positive_count = len[num for num in numbers if num>0]'
Print (positive count)
```

```
Count = 0
for i in numbers.
  if i>0
  Count = ~~Count~~ len( i in number if i>0)
  Print = ("Count")
```

```
# finding positive numbers
numbers = [10, -3, 5, 0, 8, -2]
Positive_numbers = []
for num in numbers:
  if num>0:
  Positive_numbers.append (num)
Print (positive_numbers, len (positive_numbers))
```

Activity 6.1 – Creating a Function

Title: Wrap Logic into a Function

Task

Turn average-of-marks logic into a function.

1. In words: what does the function do?

The function Calculates the average of a list of number.

2. Write code:

python

```
def average_marks(marks_list):
  total = 0
  for m in marks_list:
    total = total + m
  avg = total / len (marks list)
  return avg


data = [60, 70, 80]
print("Average =", average_marks(data))
```

3. Test with another list and note output.

$\frac{60 + 70 + 80}{3} \Rightarrow \frac{210}{3} = 70$

7.1 Discount

```
# Simple fee calculator.
Base fee = int(input("Enter base fee:"))
age = int(input("Enter age:"))
def Calculate (base fee, age):
    discount = 0.
  if age > 12:
      discount = 0.50 * base-fee
      Print (f" Discount : {discount:.2f}")
  elif age > 60:
      discount = 0.30 * base fee
      Print (f" Discount : {discount:.2f}")
  else:
      Print ("Discount : 0.00")
  return base fee - Discount
final fee = Calculate (base fee, age)
Print (f" final fee : (final fee:.2f}")
```

Activity 6.2 – Problem-Solving Template

Title: Standard Template for Any Problem — filtered avg calculator

Use this for any new problem.

1. Problem understanding
   - What is given (inputs)?

     A list of numbers and a "threshold" value
   - What is required (outputs)?

     The avg of all numbers in the list that are greater than threshold
   - Any constraints (limits, ranges)?

     The list might be empty, or no numbers may meet the threshold
2. Analysis
   - Write steps in bullets (algorithm):
     - Initialize an empty list to store "valid" numbers
     - Iterate through every number in the input list.
     - If the number is higher than the threshold, add it to the "valid" list.
3. Design
   - Will you use loops? Which? Yes, a for loop to check each number.
   - Will you use conditions (if/else)? Yes, an if statement to compare numbers to the threshold & another to check for an empty list
   - Will you use functions? Which ones? Yes, a single function named get_filtered_average.
   - What data structures (list, etc.)? A list to store the filtered values.
4. Code (Python)

python

```
# Your implementation
```

5. Test
   - Test case 1: input [10, 20, 30] (threshold 15) → output 25
   - Test case 2: input [5, 5, 5] (T=10) → output No numbers above threshold.
   - Edge case: input ~~Empty list~~ (T=0) → output No number above threshold.

- Check if the "valid" list is empty
- If not empty, divide the sum of the "valid" list by its length.
- Return the result

Activity 7.1 – Capstone Problem

Title: Simple Fee Calculator

Problem

A class charges a base fee.

1) • If student is below 12 years, 50% discount.

2) • If above 60, 30% discount.

3) • Otherwise, no discount.

Use the full template from Activity 6.2 here.

(Attach a blank copy of the template and ask students to fill all five sections.)

If - else statement.

(*). If student is below 12 Years, 50 % discount.

1) If age < 12, then discount = 0.50 × base-fee

2) Else if age > 60, then discount = 0.30 × base-fee

3) Else, then discount = 0.

final fee = base fee − discount

(6.2) Python :-

```
get_filtered average (numbers, threshould):
valid numbers = []
for n in numbers:
    if n > threshould:
        valid numbers, append (n)
if not valid numbers:
    return "No numbers above threshould."
return sum (valid numbers) / len (valid numbers
```

Example usage :-

```
data = [10, 20, 30, 40, 50]
limit = 25
Print (f"Average: {get filtered average (data, limit)}")
```

Activity 7.2 – Reflection Sheet

Title: How I Solve a New Problem

1. When I see a new problem, my first step is:
   my first step is to identify the specific factors.
2. Before coding, I make sure I know:
   - Inputs: Required data to run code.
   - Outputs: Exact answer.
3. The tools I think about (loops, if, functions) are:

4. One thing I will do differently from now on:
   Firstly I will try/write the programme on paper then I will try on Colab

(5). If/Else Statement → "My Decision maker". I use these to Categorize data (ex:- if pest count >50, then recommended actions)

Loops :- "My Heavy Lifter". I use them when I need to Perform the same Calculation over multiple rows of data

Functions :- "My Tool Box", I use these to wrap a specific process – like a B:C ratio Calculation – so I can reuse it throughout the program without rewriting it